

uni-app壁纸预览：带专属信息展示的完整实现

在壁纸预览场景中，展示每张壁纸的专属信息（如分辨率、风格标签、作者、下载量等）能帮助用户快速了解壁纸详情，提升决策效率。本文基于Swiper壁纸预览框架，新增信息展示模块，通过“固定区域+动态数据”的方式实现信息与壁纸的同步切换，同时兼顾交互流畅性与视觉美观度。

一、专属信息设计原则与核心需求

1. 设计原则

- 不遮挡核心内容**：信息区域避开壁纸主体区域，优先放置在顶部状态栏下方或底部操作栏上方；
- 与预览同步**：壁纸切换时，专属信息需无缝同步更新，避免出现“壁纸与信息不匹配”的断层；
- 层级清晰**：通过半透明背景、字体大小区分信息优先级（如风格标签为核心信息，下载量为次要信息）；
- 适配多端**：在全面屏、刘海屏设备上自动调整信息位置，避免被系统区域遮挡。

2. 核心信息维度（可按需扩展）

结合用户需求与业务场景，常见的壁纸专属信息包括：

信息类型	展示形式	优先级
风格标签	彩色标签（如“治愈系”“赛博朋克”）	高
分辨率	文本（如“2K 2560×1440”）	中
作者/来源	文本+头像（如“@设计狮小A”）	中
下载量/收藏量	图标+文本（如下载图标 + “12.5万”）	低

二、完整实现方案（含信息展示）

以“顶部信息栏+底部扩展信息”的布局形式，在原有Swiper预览框架中新增信息模块，核心实现“壁纸切换→信息同步更新”的联动效果。

1. 数据准备（新增专属信息字段）

壁纸列表数据中新增专属信息字段，确保每张壁纸的信息独立可控，列表页传递数据时一并携带。

Code block

```
1  // 示例壁纸列表数据 (含专属信息)
2  const wallpaperList = [
3    {
4      id: 1,
5      url: "https://example.com/wallpaper/1.jpg", // 高清壁纸地址
6      thumbUrl: "https://example.com/wallpaper/1_thumb.jpg", // 缩略图地址
7      info: {
8        tags: ["治愈系", "自然风光"], // 风格标签
9        resolution: "2K 2560×1440", // 分辨率
10       author: {
11         name: "@设计狮小A",
12         avatar: "https://example.com/avatar/a.jpg"
13       },
14       downloadCount: 125000, // 下载量
15       collectCount: 36800 // 收藏量
16     }
17   },
18   {
19     id: 2,
20     url: "https://example.com/wallpaper/2.jpg",
21     thumbUrl: "https://example.com/wallpaper/2_thumb.jpg",
22     info: {
23       tags: ["赛博朋克", "科技感"],
24       resolution: "4K 3840×2160",
25       author: {
26         name: "@像素极客",
27         avatar: "https://example.com/avatar/b.jpg"
28       },
29       downloadCount: 89200,
30       collectCount: 21500
31     }
32   }
33 ];
```

2. 预览页核心实现（含信息展示模块）

在Swiper容器外新增顶部信息栏，在底部操作栏内扩展详细信息区域，通过 `currentIndex` 关联当前壁纸的专属信息，实现同步切换。

2.1 页面结构（核心新增信息模块）

Code block

```
1 <template>
2   <view class="wallpaper-preview">
3
4     <swiper
5       class="preview-swiper"
6       :current="currentIndex"
7       :duration="300"
8       :circular="false"
9       @change="handleSwiperChange"
10      @touch-start="handleTouchStart"
11    >
12      <swiper-item v-for="(item, idx) in wallpaperList" :key="idx">
13        <view class="wallpaper-container">
14
15          <image
16            class="wallpaper-img thumbnail"
17            :src="item.thumbUrl"
18            mode="widthFix"
19            v-if="!item.hdLoaded"
20          ></image>
21
22
23          <image
24            class="wallpaper-img hd-img"
25            :src="item.url"
26            mode="widthFix"
27            @load="handleHdLoad(idx)"
28            v-if="idx === currentIndex || item.hdLoaded"
29            :style="{transform: `scale(${scale[idx]})`, opacity: item.hdLoaded
? 1 : 0, transition: 'opacity 0.3s ease'}"
30            @tap="handleDoubleTap(idx)"
31          ></image>
32
33
34          <view class="loading-tips" v-if="idx === currentIndex &&
!item.hdLoaded">
35            <view class="loading-spinner"></view>
36            <text class="loading-text">加载高清图中...</text>
37          </view>
38        </view>
39      </swiper-item>
40    </swiper>
41
42
43    <view class="operation-bar">
44
45      <view class="wallpaper-index">
```

```

46         {{ currentIndex + 1 }} / {{ wallpaperList.length }}
47     </view>
48
49
50     <view class="btn-group">
51         <button class="stats-btn" @click="handleCollect">
52             <button class="stats-btn" @click="handleSave">
53
54                 <button class="operate-btn set-btn" @click="handleSet">设为壁纸</button>
55             </view>
56         </view>
57     </view>
58 </template>

```

2.2 核心逻辑（信息同步与数据处理）

通过计算属性 `currentWallpaper` 关联当前索引的壁纸信息，确保切换时信息实时更新；新增数据格式化、收藏等交互逻辑。

Code block

```

1  <script>
2  import { saveImageToPhotosAlbum, setWallpaper } from '@utils/wallpaper.js';
3
4  export default {
5      data() {
6          return {
7              wallpaperList: [],    // 壁纸列表（含专属信息）
8              currentIndex: 0,      // 当前Swiper索引
9              scale: [],           // 缩放比例数组
10             statusBarHeight: 0,   // 状态栏高度（适配刘海屏）
11             collected: new Set()  // 已收藏的壁纸ID集合
12         };
13     },
14     computed: {
15         /**
16          * 核心：获取当前壁纸的专属信息
17          * 随currentIndex变化自动更新，实现信息与壁纸同步
18          */
19         currentWallpaper() {
20             return this.wallpaperList[this.currentIndex] || {};
21         }
22     },
23     onLoad(options) {
24         // 1. 接收列表页传递的参数
25         const { index, wallpaperList } = options;
26         this.currentIndex = Number(index);

```

```

27 // 解析壁纸列表 (含专属信息)
28 this.wallpaperList = JSON.parse(decodeURIComponent(wallpaperList)).map(item
=> ({
29     ...item,
30     hdLoaded: false // 高清图加载标记
31 }));
32
33 // 2. 初始化基础数据
34 this.scale = new Array(this.wallpaperList.length).fill(1);
35 // 获取状态栏高度 (适配刘海屏, 避免信息被遮挡)
36 const systemInfo = uni.getSystemInfoSync();
37 this.statusBarHeight = systemInfo.statusBarHeight;
38
39 // 3. 加载已收藏状态 (从Storage读取)
40 const collectedIds = uni.getStorageSync('wallpaper_collected');
41 if (collectedIds) {
42     this.collected = new Set(collectedIds);
43 }
44
45 // 4. 预加载当前壁纸的高清图
46 this.preloadHdImage(this.currentIndex);
47 },
48 methods: {
49     /**
50     * 预加载高清图 (原有逻辑, 保持不变)
51     */
52     preloadHdImage(idx) {
53         const item = this.wallpaperList[idx];
54         if (item && !item.hdLoaded) {
55             const img = new Image();
56             img.src = item.url;
57             img.onload = () => {
58                 this.$set(this.wallpaperList[idx], 'hdLoaded', true);
59             };
60             img.onerror = () => {
61                 uni.showToast({ title: '高清图加载失败', icon: 'none' });
62             };
63         }
64     },
65
66     /**
67     * Swiper切换事件 (核心: 同步信息与预加载)
68     */
69     handleSwiperChange(e) {
70         const newIndex = e.detail.current;
71         this.currentIndex = newIndex;
72         // 预加载当前和下一张壁纸的高清图

```

```

73     this.preloadHdImage(newIndex);
74     this.preloadHdImage(newIndex + 1);
75 },
76
77 /**
78  * 格式化大数字 (如125000 → 12.5万)
79  */
80 formatNum(num) {
81     if (num >= 10000) {
82         return (num / 10000).toFixed(1) + '万';
83     }
84     return num;
85 },
86
87 /**

```

2.3 样式设计（信息区域优化）

通过半透明背景、圆角、阴影等样式优化信息区域的视觉效果，确保既清晰可见又不遮挡壁纸内容，同时适配刘海屏设备。

Code block

```

1  <style scoped>
2  /* 预览页基础样式 (原有) */
3  .wallpaper-preview {
4      width: 100vw;
5      height: 100vh;
6      overflow: hidden;
7      background-color: #000;
8      position: relative;
9  }
10 .preview-swiper {
11     width: 100%;
12     height: 100%;
13 }
14 .wallpaper-container {
15     position: relative;
16     width: 100%;
17     height: 100%;
18     display: flex;
19     align-items: center;
20     justify-content: center;
21 }
22 .wallpaper-img {
23     width: 100%;
24     height: auto;
25     object-fit: contain;

```

```
26     position: absolute;
27     top: 50%;
28     left: 50%;
29     transform: translate(-50%, -50%);
30 }
31 .hd-img {
32     opacity: 0;
33     z-index: 10;
34 }
35 .thumbnail {
36     z-index: 1;
37     filter: blur(2px);
38 }
39 .loading-tips {
40     position: absolute;
41     top: 50%;
42     left: 50%;
43     transform: translate(-50%, -50%);
44     z-index: 10;
45     color: #fff;
46     text-align: center;
47 }
48 .loading-spinner {
49     width: 40rpx;
50     height: 40rpx;
51     border: 4rpx solid rgba(255,255,255,0.3);
52     border-top-color: #fff;
53     border-radius: 50%;
54     animation: spin 1s linear infinite;
55     margin-bottom: 16rpx;
56 }
57 @keyframes spin {
58     0% { transform: rotate(0deg); }
59     100% { transform: rotate(360deg); }
60 }
61
62 /* 新增：顶部信息栏样式 */
63 .top-info-bar {
64     position: fixed;
65     top: 0;
66     left: 0;
67     width: 100%;
68     padding: 16rpx 30rpx;
69     box-sizing: border-box;
70     display: flex;
71     justify-content: space-between;
72     align-items: center;
```

```
73     z-index: 20;
74     /* 半透明背景，不遮挡壁纸 */
75     background: linear-gradient(rgba(0,0,0,0.5), transparent);
76 }
77 /* 风格标签组 */
78 .tag-group {
79     display: flex;
80     gap: 12rpx;
81     flex-wrap: wrap;
82 }
83 .tag {
84     padding: 6rpx 16rpx;
85     background-color: rgba(44, 131, 255, 0.7);
86     color: #fff;
87     font-size: 22rpx;
88     border-radius: 20rpx;
```

三、关键优化点与注意事项

1. 信息与壁纸同步的核心保障

- **计算属性联动**：通过 `currentWallpaper` 计算属性直接关联 `currentIndex`，无需手动更新信息，确保切换时“壁纸动、信息动”的同步性；
- **数据完整性校验**：在 `currentWallpaper` 中添加默认值（如 `|| {}`），避免壁纸列表为空时页面报错；
- **预加载与信息同步**：预加载壁纸时不影响信息展示，即使高清图未加载完成，专属信息仍能通过缩略图页快速呈现。

2. 视觉体验优化技巧

1. **半透明背景分层**：信息区域使用 `linear-gradient` 渐变背景，从顶部/底部向中间透明过渡，既突出信息又不割裂壁纸视觉；
2. **优先级排版**：高优先级信息（标签、分辨率）放在顶部易见区域，次要信息（下载量、作者）放在底部，符合用户视觉浏览习惯；
3. **动态适配刘海屏**：通过 `statusBarHeight` 动态设置顶部信息栏的 `paddingTop`，避免信息被刘海遮挡，适配所有全面屏设备。

3. 性能与交互保障

- **避免信息区域遮挡交互**：顶部信息栏高度控制在80rpx以内，底部操作栏不超过120rpx，确保壁纸主体区域（中间80%）可正常响应双击缩放等手势；
- **大数字格式化**：通过 `formatNum` 方法将“125000”转为“12.5万”，避免长数字占用过多空间，提升视觉简洁性；

- **图片懒加载**：作者头像等小图添加 `lazy-load` 属性，减少首次加载的资源开销。

4. 扩展性设计

若需新增更多专属信息（如壁纸描述、使用场景建议），可通过以下方式扩展：

Code block

```
1  // 新增壁纸描述信息（示例）
2  <view class="wallpaper-desc" v-if="currentWallpaper.info.desc">
3    {{ currentWallpaper.info.desc }}
4  </view>
5
6  // 对应样式
7  .wallpaper-desc {
8    position: fixed;
9    top: 120rpx;
10   left: 50%;
11   transform: translateX(-50%);
12   width: 80%;
13   color: #fff;
14   font-size: 24rpx;
15   background-color: rgba(0,0,0,0.4);
16   padding: 16rpx;
17   border-radius: 12rpx;
18   z-index: 20;
19   text-align: center;
20 }
```

四、总结

在壁纸预览中展示专属信息，核心是通过“数据联动+合理布局”实现信息与壁纸的精准匹配。本文通过顶部信息栏展示高优先级标签与分辨率，底部操作栏扩展作者、下载量等细节信息，既满足了用户的信息获取需求，又通过视觉优化确保不影响壁纸预览的核心体验。同时，通过计算属性、动态适配等技术手段，保障了功能的稳定性与多端兼容性。实际开发中可根据业务需求，灵活增减信息维度，进一步提升壁纸预览功能的实用性与竞争力。

（注：文档部分内容可能由 AI 生成）